Linux Application Development - Day 2

Debugging, System Calls & Memory Management

Instructor: [Your Name]

Duration: 8 hours (including labs)

Welcome Back!

Yesterday's Recap

✅ Compilation pipeline and GCC
✅ Static and shared libraries
✅ Make build systems
✅ Git version control

Today's Journey

🔍 **Debug like a pro** - GDB, Valgrind, core dumps
🔧 **Understand the kernel boundary** - System calls
💾 **Master memory** - Virtual memory, heap, stack

Day 2 Agenda

Time	Topic
09:00-10:30	GDB Debugging Fundamentals
10:30-10:45	Break
10:45-12:00	Core Dumps & Debugging Tools
12:00-13:00	Lunch
13:00-14:30	System Calls & Kernel Interface
14:30-14:45	Break
14:45-16:00	Memory Management
16:00-17:00	Hands-On Lab

Section 1: GDB - The GNU Debugger

Why Learn GDB?

Professional Debugging Skills

- **printf debugging is limited** - can't inspect live state
- **Production bugs** - reproduce issues in controlled environment
- **Understand crashes** - what happened before SIGSEGV?
- **Reverse engineer** - understand unfamiliar code
- **Optimize** - find performance bottlenecks

GDB is THE standard debugger for Linux C/C++

GDB Workflow Overview

Compiling for Debugging

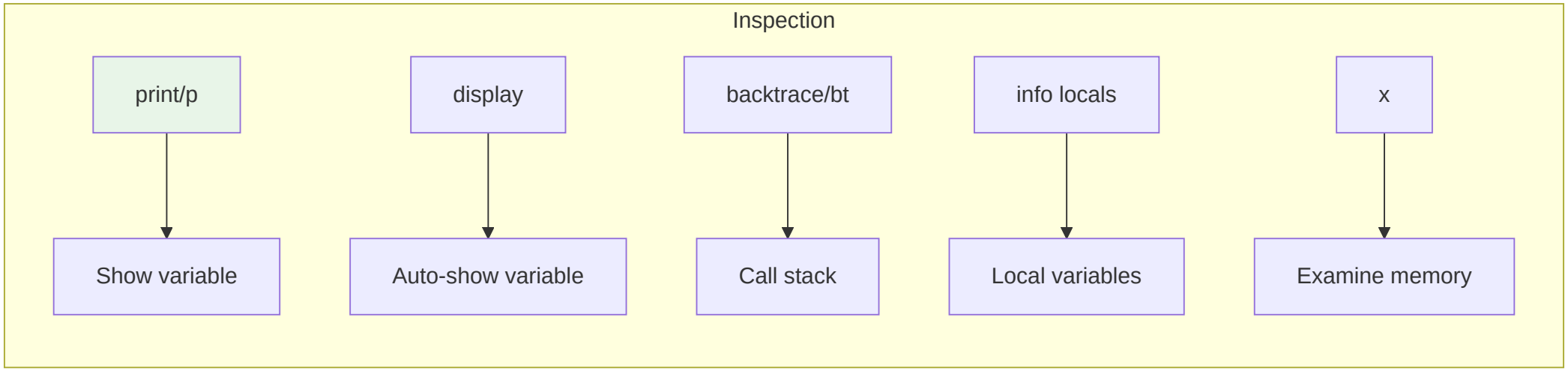
```
# Debug build: -g adds symbols, -O0 disables optimization
gcc -g -O0 -Wall program.c -o program

# What -g does:
# - Adds symbol table (function names, variables)
# - Source line number mapping
# - Type information for variables
# - Makes debugging human-readable

# Release build (after debugging):
gcc -O2 -Wall program.c -o program
# Symbols stripped, optimizations enabled
```

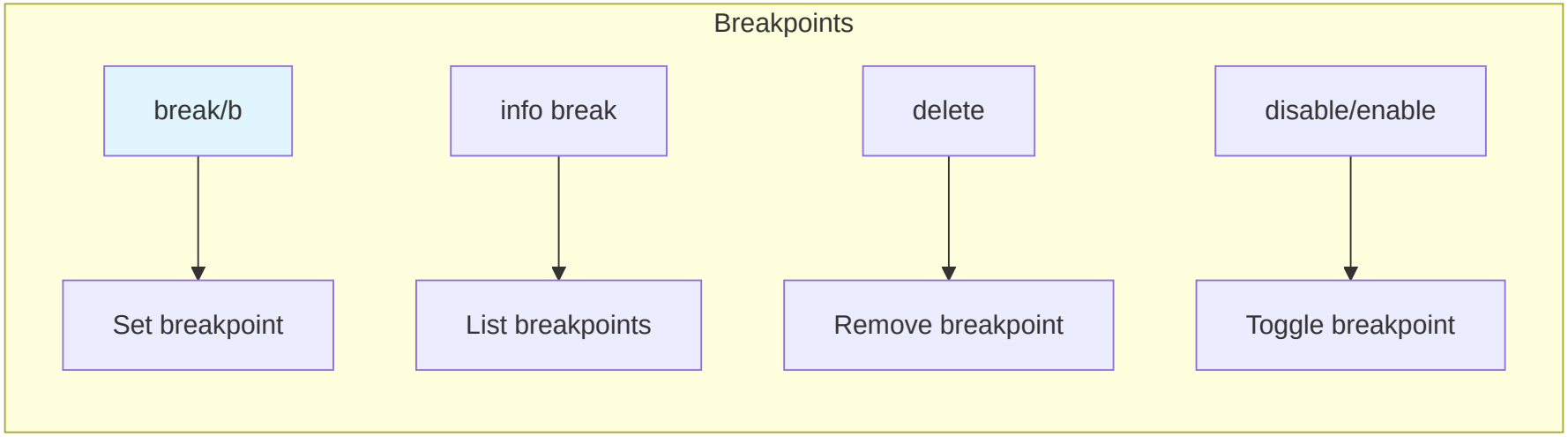
Always use `-g -O0` for debugging builds

Starting GDB

```
# Launch GDB with program
gdb ./program

# With arguments
gdb --args ./program arg1 arg2

# Attach to running process
gdb -p 1234

# Non-interactive mode (for scripts)
gdb -batch -ex "bt" -ex "quit" ./program core

# GDB with pretty printing
gdb -tui ./program  # Text User Interface
```

Essential GDB Commands





Execution Control

```
(gdb) run                  # Start program (or 'r')
(gdb) run arg1 arg2        # With arguments
(gdb) continue             # Resume until next breakpoint (or 'c')
(gdb) next                 # Step over function calls (or 'n')
(gdb) step                 # Step into function calls (or 's')
(gdb) finish               # Run until current function returns
(gdb) until 42             # Run until line 42
(gdb) stepi                # Step one CPU instruction
(gdb) nexti                # Step over one instruction

# Ctrl-C: Interrupt running program
```

Setting Breakpoints

```
# Breakpoint at function
(gdb) break main          # or 'b main'
(gdb) break calculate_sum

# Breakpoint at specific line
(gdb) break file.c:25
(gdb) break 42            # Line in current file

# Conditional breakpoints
(gdb) break main if argc > 2
(gdb) break process_data if size == 0

# Breakpoint on memory write
(gdb) watch my_variable   # Watchpoint
(gdb) rwatch ptr          # Read watchpoint

# Manage breakpoints
(gdb) info breakpoints
(gdb) delete 1            # Delete breakpoint #1
(gdb) disable 2           # Temporarily disable
(gdb) enable 2
```

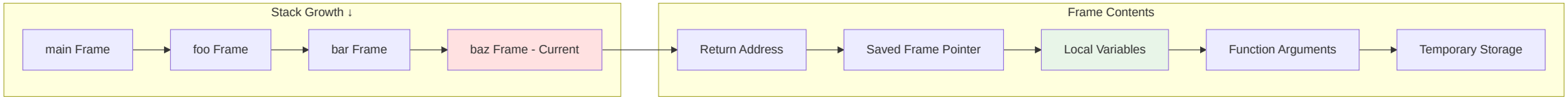
Inspecting Variables

```
# Print variable
(gdb) print variable_name    # or 'p'
(gdb) print *ptr             # Dereference pointer
(gdb) print array[5]
(gdb) print struct.member

# Display format
(gdb) print/x value          # Hexadecimal
(gdb) print/d value          # Decimal
(gdb) print/t value          # Binary
(gdb) print/c value          # Character
(gdb) print/ value          # Format: x d o t a c f s

# Automatic display
(gdb) display variable       # Show at every step
(gdb) info display
(gdb) undisplay 1
```

Examining Memory

```
# Examine memory at address
(gdb) x/10x $sp            # 10 hex words from stack pointer
(gdb) x/20c 0x400080       # 20 chars from address
(gdb) x/10i main           # 10 instructions from main

# Format: x/[count][format][size] address
# Format: x (hex), d (decimal), u (unsigned), t (binary),
#         o (octal), a (address), c (char), s (string), i (instruction)
# Size: b (byte), h (halfword), w (word), g (giant/8-bytes)

# Example
(gdb) x/4xw ptr            # 4 hex words
```

Stack Traces (Backtraces)



Navigating the Stack

```
# View call stack
(gdb) backtrace             # or 'bt'
(gdb) backtrace full        # With local variables
(gdb) bt 5                  # Only 5 frames

# Select frame
(gdb) frame 0               # Bottom frame (current)
(gdb) frame 2               # Move up 2 frames
(gdb) up                    # Move to calling frame
(gdb) down                  # Move to called frame

# Frame information
(gdb) info frame
(gdb) info locals           # Local variables in current frame
(gdb) info args             # Function arguments
(gdb) info registers        # CPU registers
```

Critical for understanding program flow

Real GDB Session Example

```
$ gdb ./crash
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x00000000004005e7 in process_data (data=0x0) at main.c:42
42          result = data->value * 2;

(gdb) backtrace
#0  0x00000000004005e7 in process_data (data=0x0) at main.c:42
#1  0x0000000000400623 in calculate (items=0x7fff8c80) at main.c:58
#2  0x0000000000400698 in main (argc=2, argv=0x7fff8e88) at main.c:78

(gdb) frame 1
#1  0x0000000000400623 in calculate (items=0x7fff8c80) at main.c:58
58          process_data(items[i].data);

(gdb) print i
$1 = 5
(gdb) print items[i].data
$2 = (struct data *) 0x0
```

Core Dumps: Post-Mortem Analysis

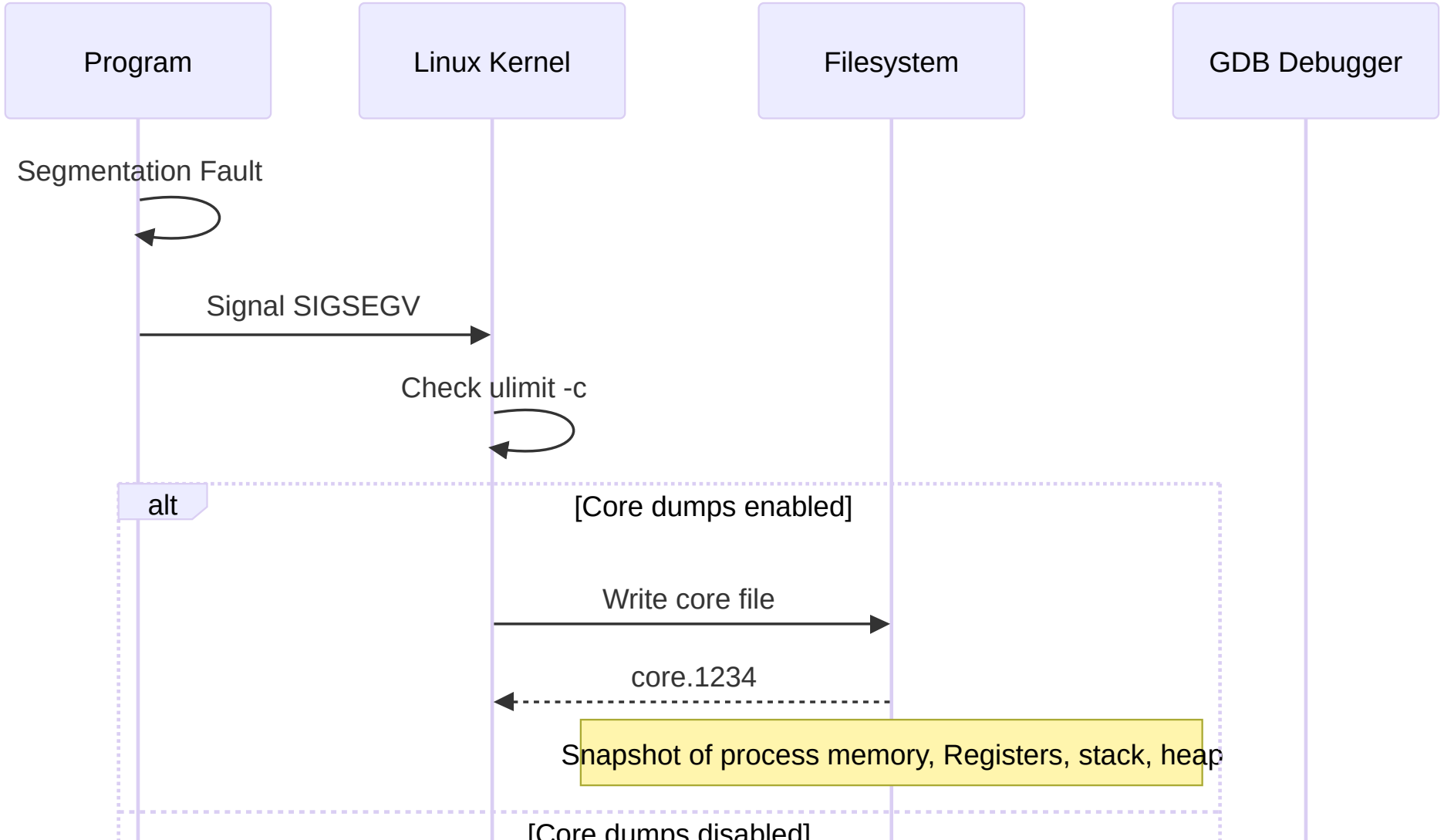
What is a Core Dump?

Snapshot of Crashed Process

- **Memory contents** at time of crash
- **Register states** (PC, SP, etc.)
- **Open file descriptors**
- **Stack traces** for all threads

Like a "photograph" of a crime scene

Core Dump Analysis Flow



Enabling Core Dumps

```
# Check current limit
ulimit -c

# Enable unlimited core dumps
ulimit -c unlimited

# Make permanent (add to ~/.bashrc)
echo "ulimit -c unlimited" >> ~/.bashrc

# System-wide configuration
sudo sysctl -w kernel.core_pattern=/var/crash/core.%e.%p.%t
# %e = executable name
# %p = process ID
# %t = timestamp
# %s = signal number

# View current pattern
cat /proc/sys/kernel/core_pattern
```

Generating Core Dumps

```
# Method 1: Let program crash
./buggy_program
# Segmentation fault (core dumped)

# Method 2: Send SIGABRT
kill -ABRT <pid>

# Method 3: From within GDB
(gdb) generate-core-file
# Saved corefile core.12345

# Method 4: Capture running process
gcore <pid>
# Saved corefile core.12345 (non-destructive)
```

Analyzing Core Dumps

```
# Load core dump in GDB
gdb ./program core.1234

# Immediately see crash point
(gdb) bt
#0  0x00007f8b9c4e7f7f in some_function () at file.c:42
#1  0x00007f8b9c4e8123 in main () at main.c:18

# Inspect variables at crash
(gdb) frame 0
(gdb) info locals
(gdb) print problematic_var
(gdb) x/20x $sp

# Check all threads (multi-threaded programs)
(gdb) info threads
(gdb) thread 2
(gdb) bt
```

Core Dump Best Practices

Development

✅ Always enable during development: `ulimit -c unlimited`
✅ Compile with `-g` to get meaningful symbols
✅ Keep debug binaries matching production
✅ Store cores with version information

Production

✅ Configure `/proc/sys/kernel/core_pattern` to central location
✅ Monitor disk space (cores can be huge)
✅ Implement automatic core analysis
⚠️ Consider security (cores contain passwords in memory)

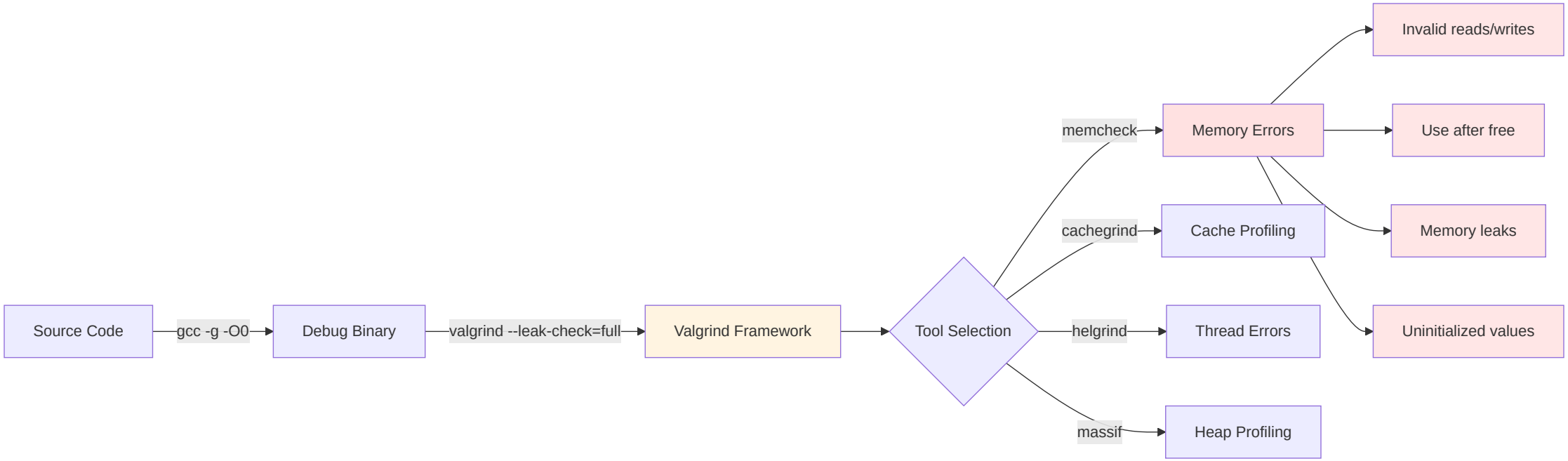
Section 2: Advanced Debugging Tools

Valgrind: Memory Error Detection

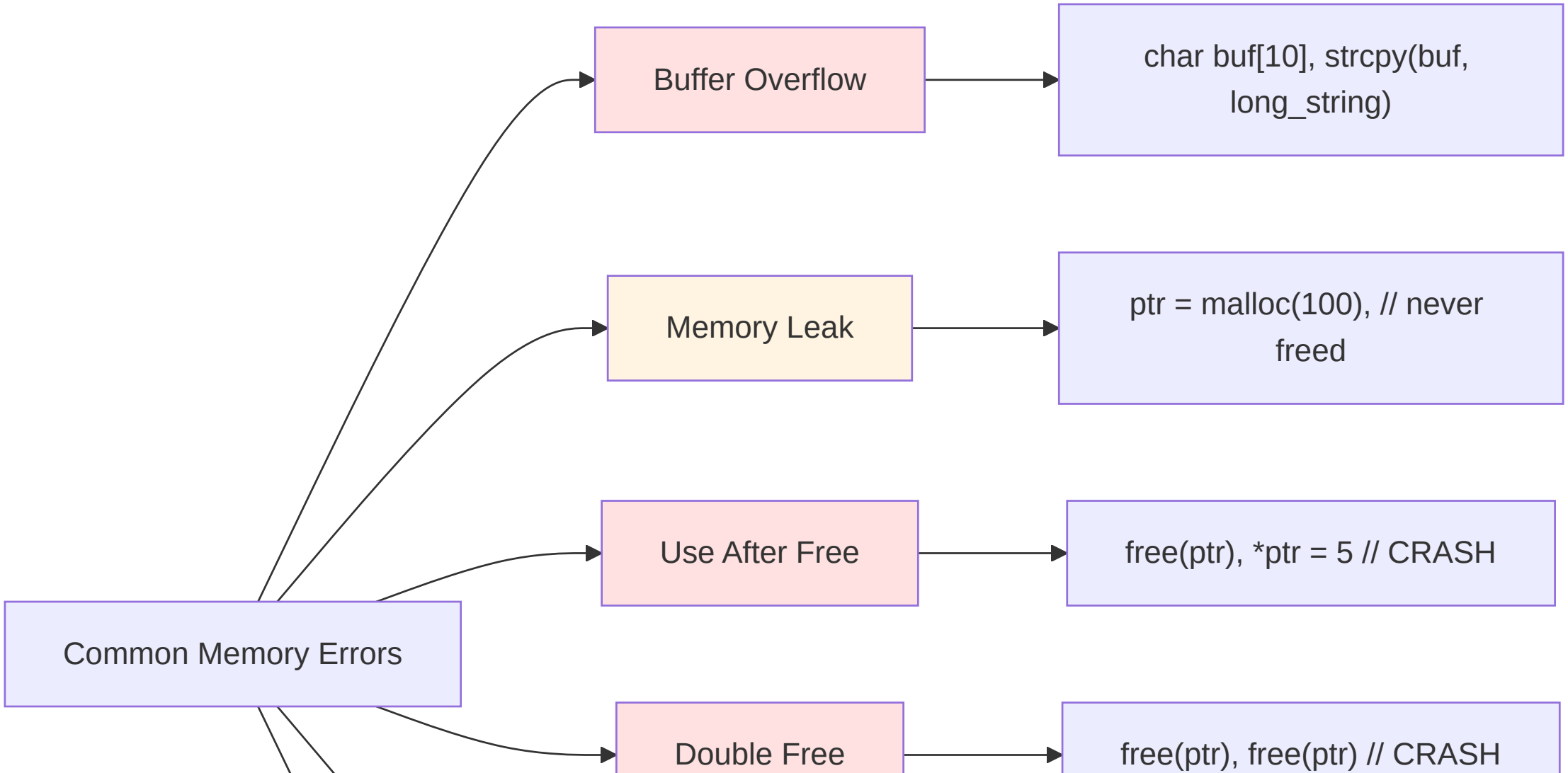
The Swiss Army Knife of Debugging

```
# Install valgrind
sudo apt install valgrind   # Debian/Ubuntu
sudo yum install valgrind   # RHEL/CentOS

# Basic memcheck
valgrind ./program

# Detailed leak check
valgrind --leak-check=full --show-leak-kinds=all ./program

# Track origins of uninitialized values
valgrind --track-origins=yes ./program
```

Valgrind Workflow



Common Memory Errors Detected



Valgrind Output Example

```
$ valgrind --leak-check=full ./leaky
==12345== Memcheck, a memory error detector
==12345== Invalid write of size 4
==12345==    at 0x40053E: main (leak.c:10)
==12345==  Address 0x52050a0 is 0 bytes after a block of size 40 alloc'd
==12345==    at 0x4C2DB8F: malloc (in /usr/lib/valgrind/...)
==12345==    by 0x400530: main (leak.c:8)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 100 bytes in 1 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==
==12345== ERROR SUMMARY: 2 errors from 2 contexts
```

Valgrind Tools Suite

memcheck (default)

- Memory leaks, invalid reads/writes
- Use of uninitialized memory
- **Most commonly used**

cachegrind

- Cache simulation, profiling
- L1/L2 cache misses
- Branch prediction analysis

callgrind

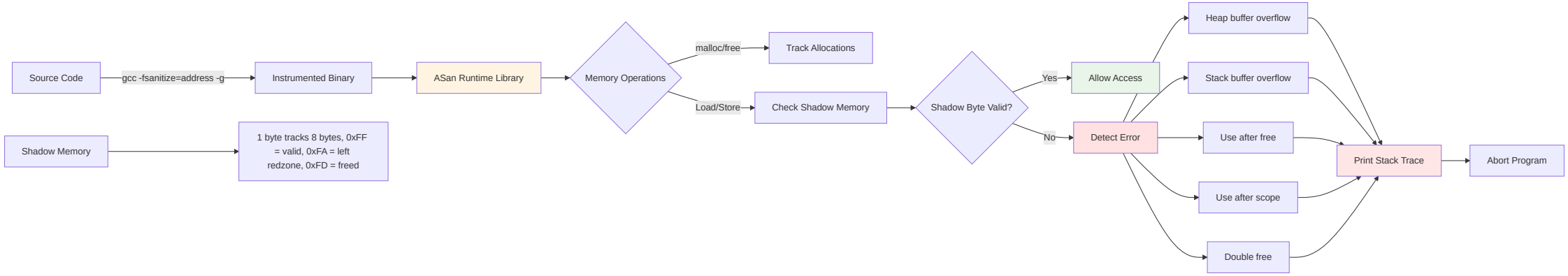
AddressSanitizer (ASan)

Modern Alternative to Valgrind

```
# Compile with ASan
gcc -fsanitize=address -g program.c -o program

# Run normally (ASan runtime automatically loaded)
./program

# Detects:
# - Heap buffer overflow
# - Stack buffer overflow
# - Use after free
# - Use after return
# - Double free
```

~2x slowdown vs. Valgrind's 20-50x

AddressSanitizer Architecture

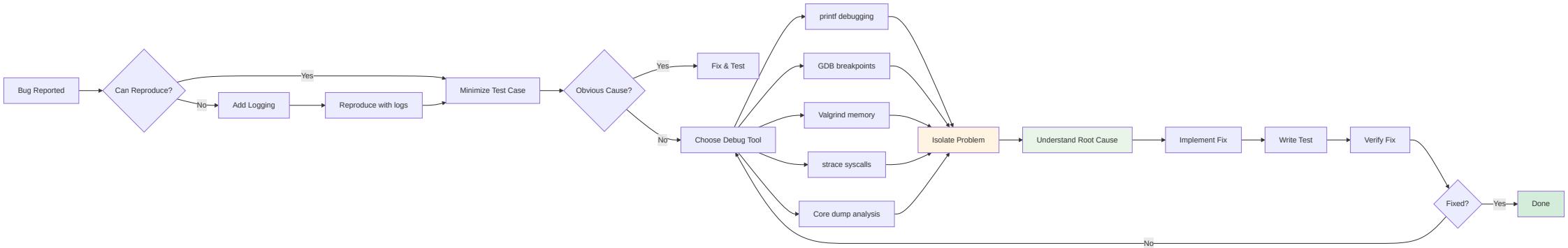


ASan Output Example

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x60300000eff4
READ of size 4 at 0x60300000eff4 thread T0
    #0 0x4008d0 in main /home/user/test.c:10
    #1 0x7f89e4c2e830 in __libc_start_main (/lib/x86_64.../libc.so.6+0x20830)
    #2 0x400779 in _start (/home/user/test+0x400779)

0x60300000eff4 is located 0 bytes to the right of 20-byte region [0x60300000efe0,0x60300000eff4)
allocated by thread T0 here:
    #0 0x7f89e532eb50 in __interceptor_malloc (/usr/lib/.../libasan.so.4+0xdeb50)
    #1 0x400897 in main /home/user/test.c:8
```

Immediate stack trace of error AND allocation

Debugging Strategy Flowchart



Section 3: System Calls

What is a System Call?

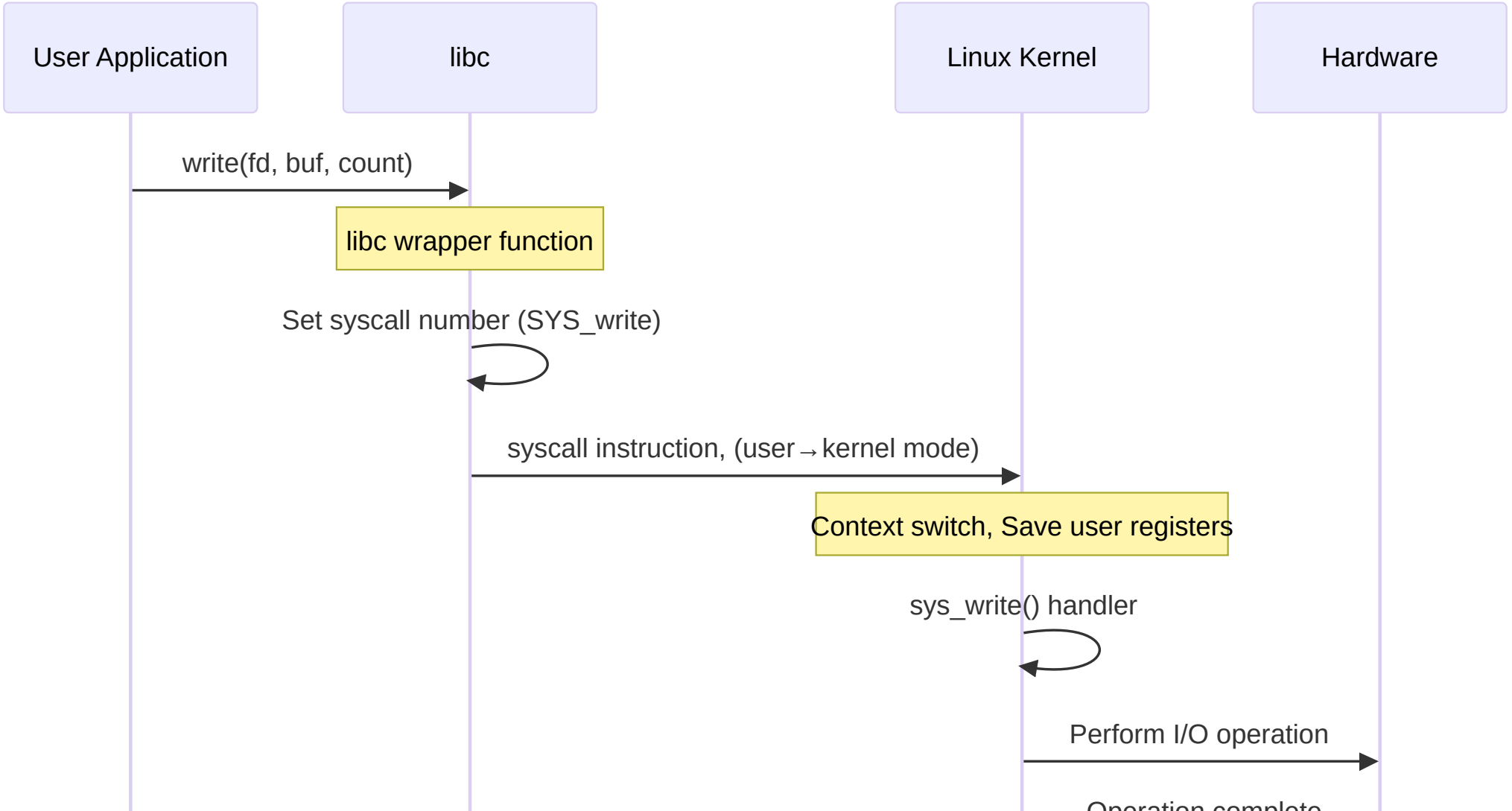
The Kernel Gateway

- **User space** - your application code (unprivileged)
- **Kernel space** - OS code (privileged, Ring 0)
- **System calls** - controlled entry points to kernel

You CANNOT access hardware directly in user space

```
// User code
int fd = open("/dev/sda", O_RDONLY);  // System call!
// Kernel handles actual disk I/O
```

System Call Flow



System Call vs Library Function

System Calls (kernel operations)

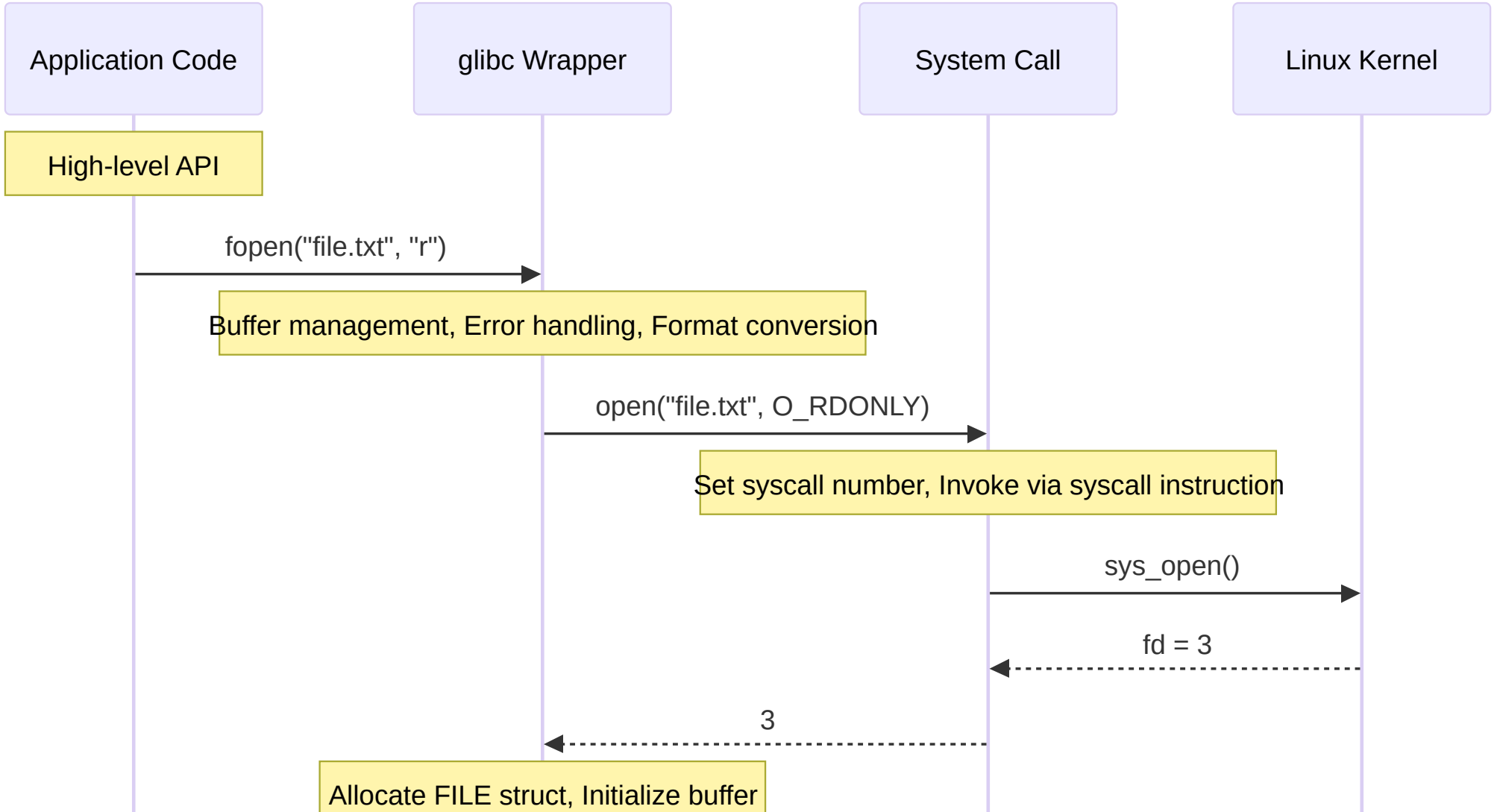
```
open(), read(), write(), close()
fork(), exec(), exit(), wait()
mmap(), mprotect(), brk()
socket(), bind(), listen()
```

Library Functions (user space)

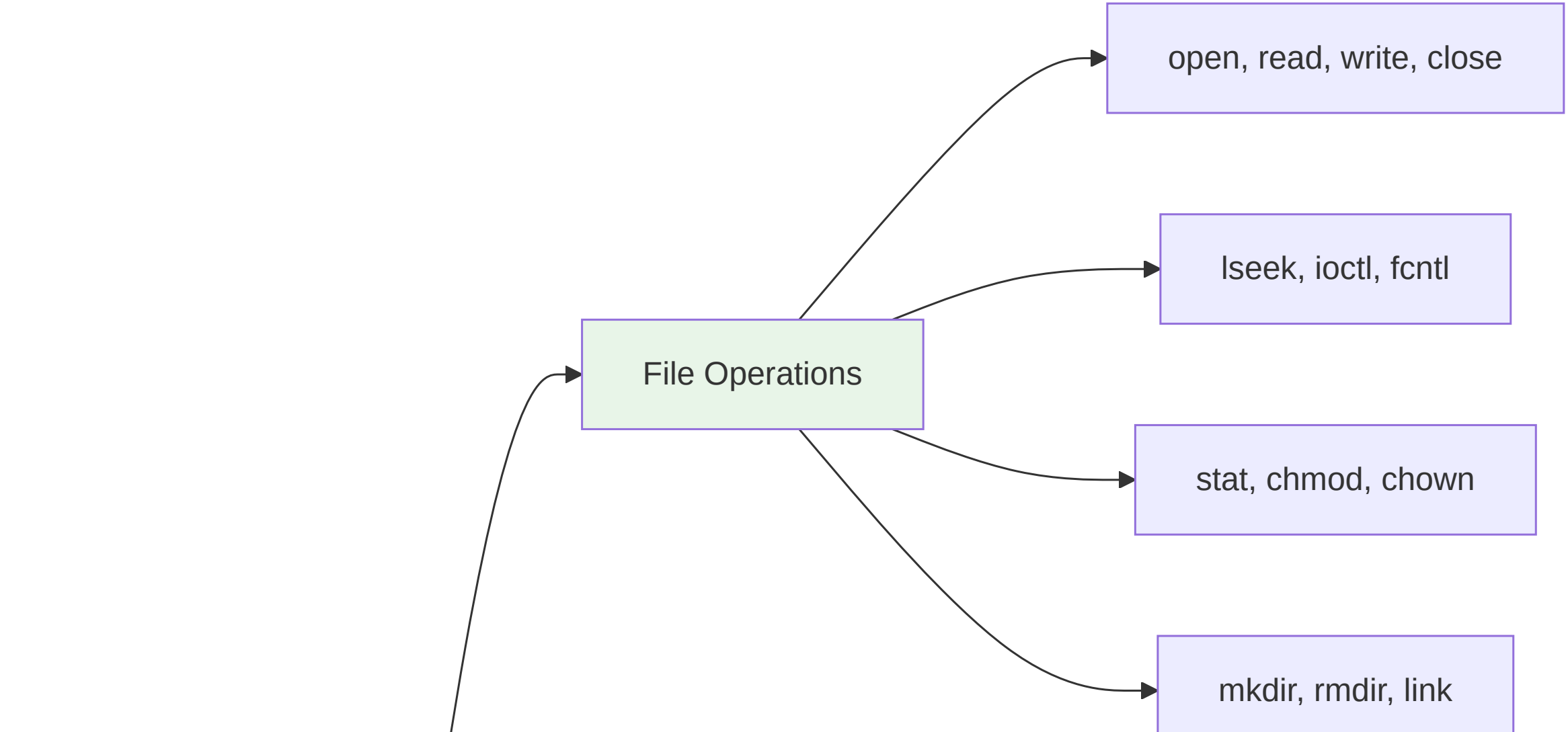
```
fopen(), fread(), fwrite(), fclose()  // Buffered I/O
printf(), scanf()                      // Formatted I/O
malloc(), free()                       // Memory management
strlen(), strcpy()                     // String operations
```

Library functions often wrap system calls

Wrapper Functions



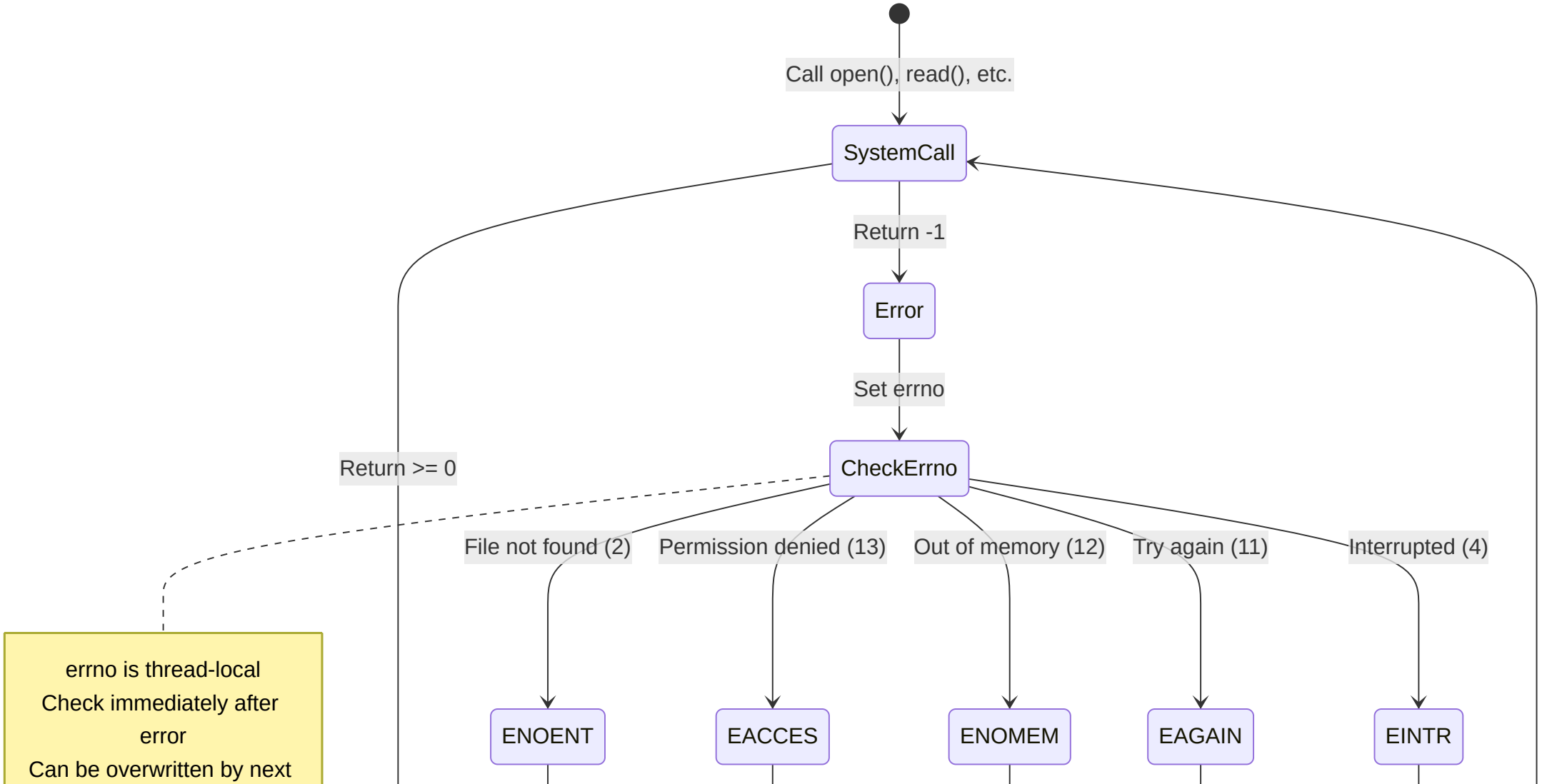
System Call Categories



Making System Calls

```c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <errno.h>

int main() {
    // System call: open
    int fd = open("/etc/passwd", O_RDONLY);
    if (fd == -1) {
        perror("open");  // Print error message
        return 1;
    }

    // System call: read
    char buffer[1024];
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes_read == -1) {
        perror("read");
        close(fd);
        return 1;
    }

    buffer[bytes_read] = '\0';

    // System call: write (stdout)
    write(STDOUT_FILENO, buffer, bytes_read);

    // System call: close
    close(fd);
    return 0;
}
```

Error Handling with errno



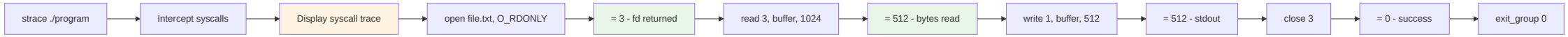
Common errno Values

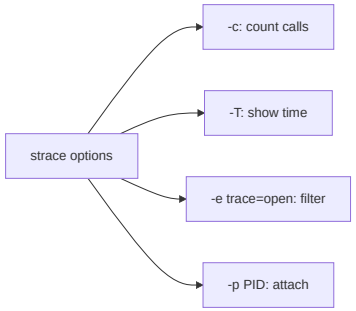
```c
#include <errno.h>
#include <string.h>

// Common error codes
ENOENT  (2)   // No such file or directory
EACCES  (13)  // Permission denied
EINVAL  (22)  // Invalid argument
ENOMEM  (12)  // Out of memory
EAGAIN  (11)  // Try again (non-blocking I/O)
EINTR   (4)   // Interrupted system call
EBADF   (9)   // Bad file descriptor
EEXIST  (17)  // File exists
EPIPE   (32)  // Broken pipe

// Using errno
int fd = open("file.txt", O_RDONLY);
if (fd == -1) {
    printf("Error %d: %s\n", errno, strerror(errno));
    // Or simply:
    perror("open");
}
```

Proper Error Handling Pattern

```c
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>

ssize_t safe_write(int fd, const void *buf, size_t count) {
    ssize_t written = 0;
    ssize_t result;

    while (written < count) {
        result = write(fd, buf + written, count - written);

        if (result == -1) {
            if (errno == EINTR) {
                // Interrupted by signal, retry
                continue;
            }
            // Real error
            return -1;
        }

        written += result;
    }

    return written;
}
```

strace: Tracing System Calls





Using strace

```
# Trace all system calls
strace ./program

# Trace specific syscalls
strace -e trace=open,read,write ./program

# Count syscalls
strace -c ./program

# Trace with timestamps
strace -t ./program
strace -T ./program  # With time spent in each

# Attach to running process
strace -p 1234

# Save to file
strace -o trace.log ./program

# Follow forks
strace -f ./program
```

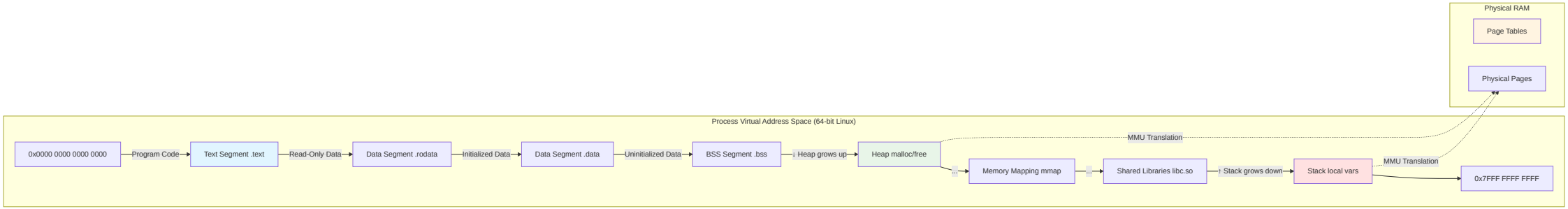
strace Example Output

```
$ strace cat /etc/hostname
execve("/bin/cat", ["cat", "/etc/hostname"], [/* 67 vars */]) = 0
brk(NULL)                                = 0x55e9a7d4b000
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=9, ...}) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f89e4e4a000
read(3, "myhost\n", 131072)              = 7
write(1, "myhost\n", 7myhost
)                  = 7
read(3, "", 131072)                      = 0
close(3)                                 = 0
exit_group(0)                            = ?
+++ exited with 0 +++
```

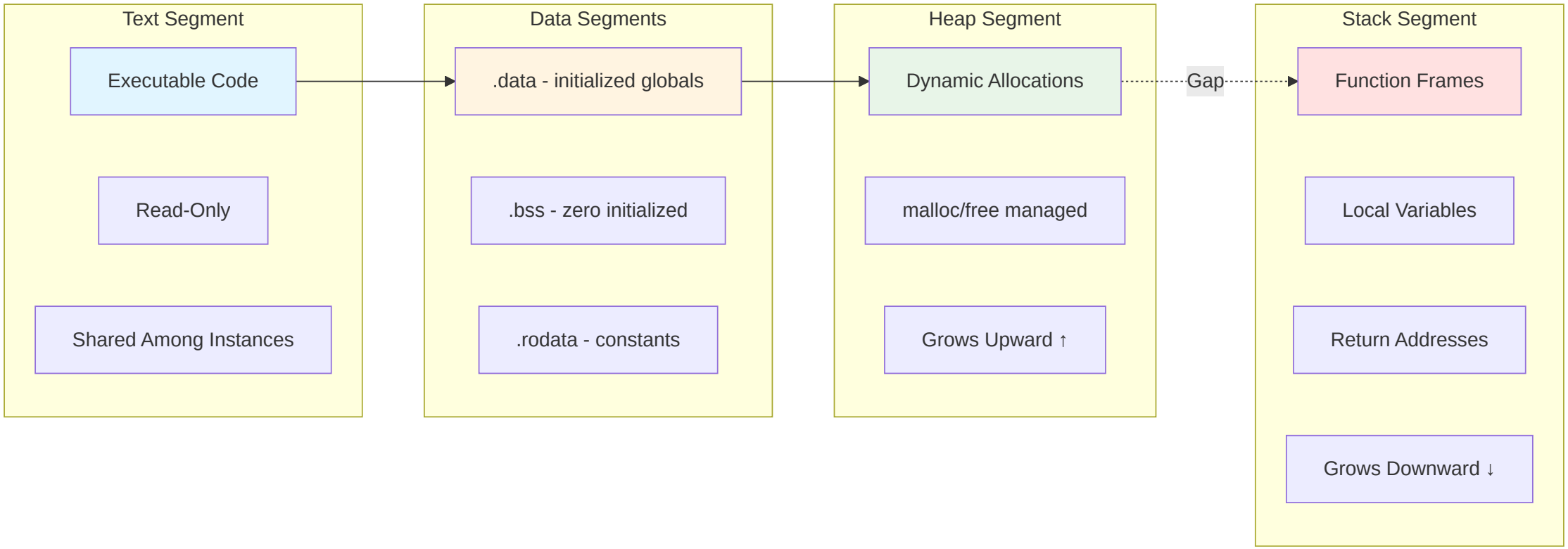
See exactly what your program does at kernel level

Section 4: Memory Management

Virtual Memory Architecture



Memory Layout of a Process



Memory Segments Explained

Text Segment (.text)

- Executable code
- Read-only, shared among instances
- Mapped from executable file

Data Segment (.data, .bss)

- **. data** - Initialized global/static variables
- **.bss** - Uninitialized global/static (zeroed)

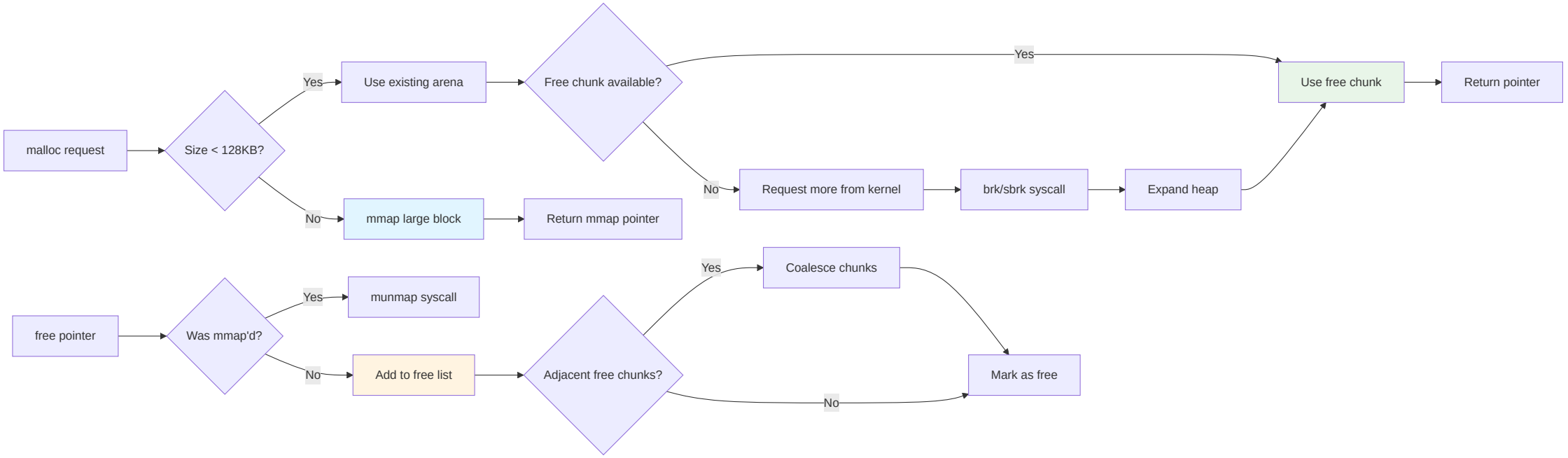
Heap

- Dynamic allocations (malloc/free)

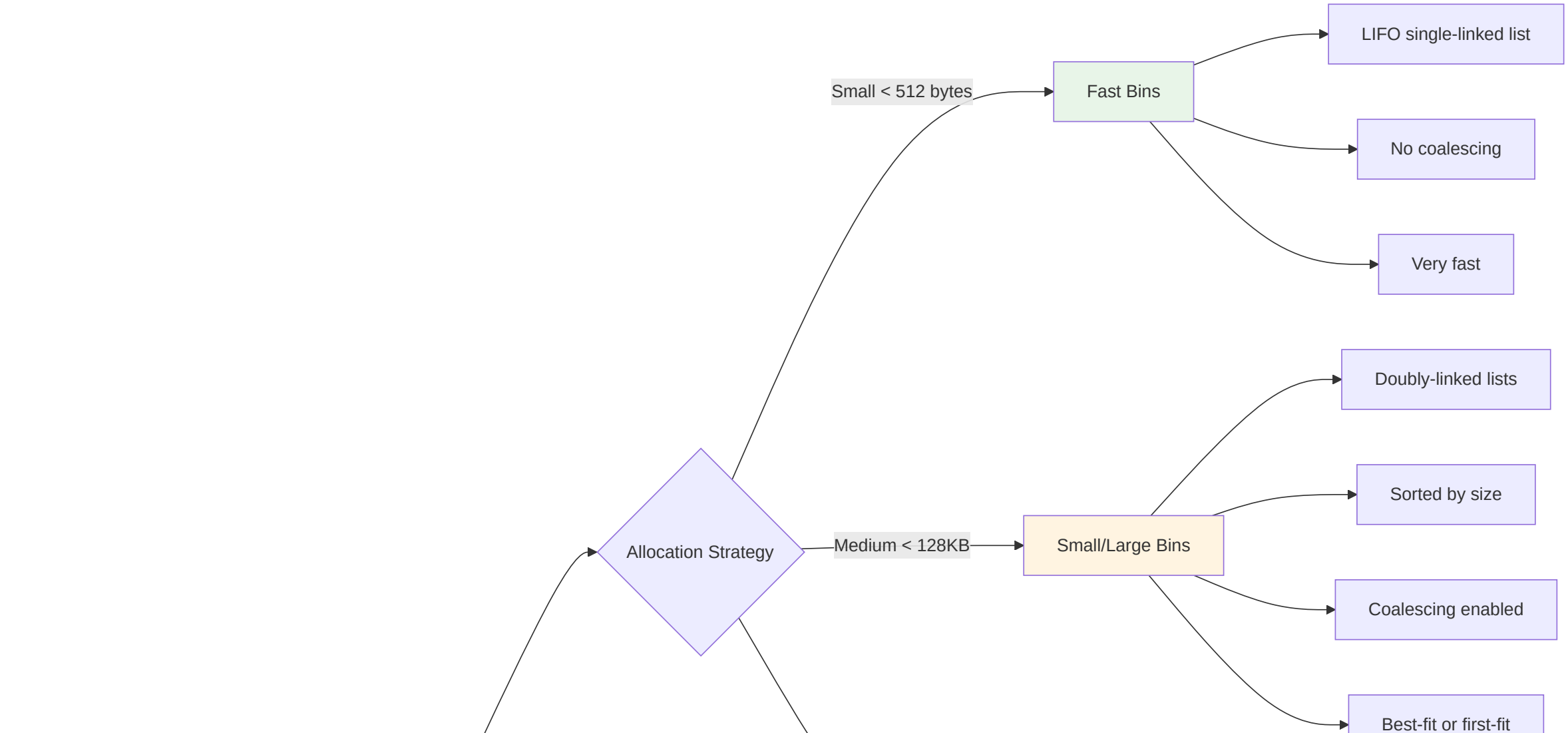
Dynamic Memory Allocation

```c
#include <stdlib.h>
#include <string.h>

// malloc: allocate uninitialized memory
void *malloc(size_t size);
int *arr = malloc(10 * sizeof(int));
if (arr == NULL) {
    perror("malloc");
    exit(1);
}

// calloc: allocate zero-initialized memory
void *calloc(size_t nmemb, size_t size);
int *arr = calloc(10, sizeof(int));  // All zeros

// realloc: resize allocation
void *realloc(void *ptr, size_t size);
arr = realloc(arr, 20 * sizeof(int));

// free: deallocate memory
void free(void *ptr);
free(arr);
arr = NULL;  // Good practice!
```

Heap Allocator Architecture



malloc Implementation Details



Memory Allocation Pitfalls

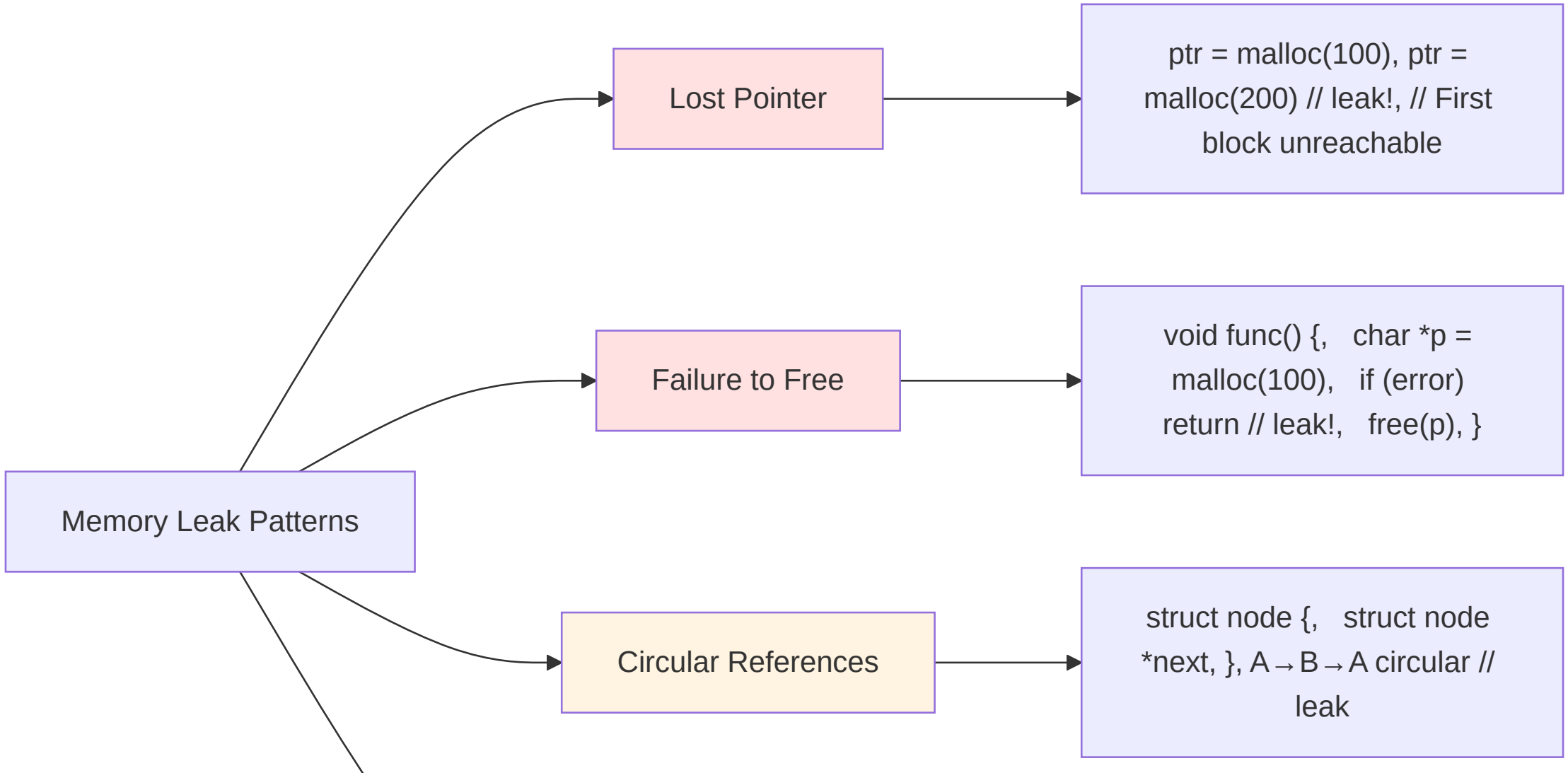
Memory Leaks

```
void leak_example() {
    char *data = malloc(1000);
    // ... use data ...
    if (error_condition) {
        return;  // LEAK! Forgot to free()
    }
    free(data);
}
```

Use After Free

```
char *ptr = malloc(100);
free(ptr);
strcpy(ptr, "hello");  // CRASH! Undefined behavior
```

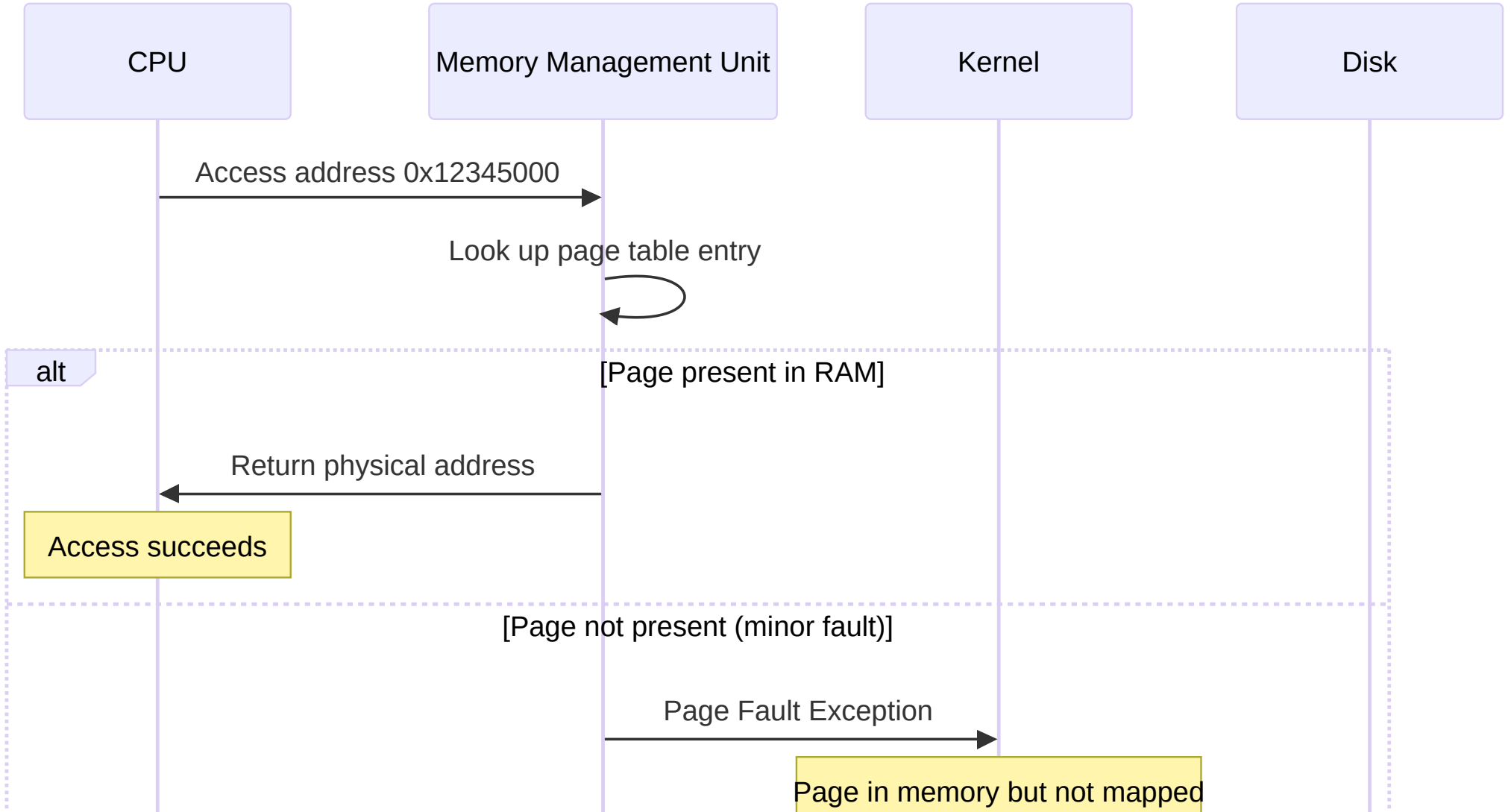
Memory Leak Patterns



Proper Memory Management

```c
// Pattern 1: Always pair malloc/free
char *data = malloc(size);
if (data == NULL) {
    return -ENOMEM;
}
// ... use data ...
free(data);
data = NULL;

// Pattern 2: RAII-style with goto
char *a = NULL, *b = NULL;
int fd = -1;

a = malloc(100);
if (!a) goto cleanup;

fd = open("file", O_RDONLY);
if (fd == -1) goto cleanup;

b = malloc(200);
if (!b) goto cleanup;

// ... work ...

cleanup:
    free(b);
    free(a);
    if (fd != -1) close(fd);
    return status;
```

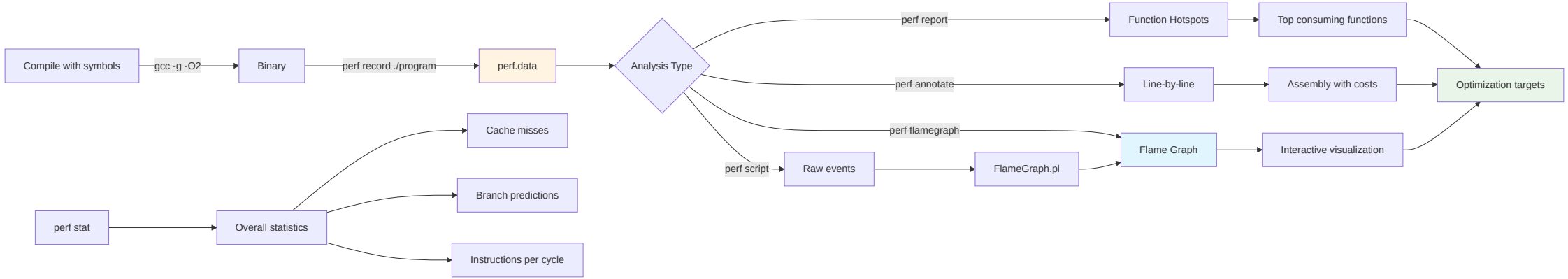
Page Faults and Virtual Memory



Memory Mapping with mmap()

```c
#include <sys/mman.h>
#include <fcntl.h>

// Map file into memory
int fd = open("data.bin", O_RDONLY);
void *addr = mmap(NULL, file_size, PROT_READ,
                  MAP_PRIVATE, fd, 0);
if (addr == MAP_FAILED) {
    perror("mmap");
    return -1;
}

// Access file contents as memory
unsigned char *data = (unsigned char *)addr;
printf("First byte: %02x\n", data[0]);

// Unmap when done
munmap(addr, file_size);
close(fd);
```

#Performance Profiling with perf



Using perf

```
# Record performance data
perf record ./program

# View report
perf report

# Annotate source with performance data
perf annotate

# Real-time monitoring
perf top

# Count events
perf stat ./program

# Record with call graph
perf record -g ./program

# Generate flame graph
perf script | ./FlameGraph/stackcollapse-perf.pl | \
  ./FlameGraph/flamegraph.pl > flame.svg
```

Memory Performance Tips

Do's

✅ Prefer stack allocation when possible
✅ Reuse allocations instead of repeated malloc/free
✅ Use memory pools for fixed-size objects
✅ Align data structures to cache lines (64 bytes)
✅ Profile before optimizing

Don'ts

❌ Allocate in tight loops
❌ Leak memory (even "small" leaks compound)
❌ Ignore alignment for performance-critical data
❌ Use malloc for tiny allocations (<16 bytes)

Advanced Memory Tools

Electric Fence

```
# Link with Electric Fence
gcc -g program.c -o program -lefence
./program
# Crashes immediately on buffer overflow
```

Massif (Heap Profiler)

```
valgrind --tool=massif ./program
ms_print massif.out.12345
# Shows memory usage over time
```

mallopt() Tuning

Day 2 Summary

What We Covered

✅ **GDB debugging** - Breakpoints, inspection, backtraces
✅ **Core dumps** - Post-mortem analysis of crashes
✅ **Valgrind & ASan** - Memory error detection
✅ **System calls** - Kernel interface, errno handling, strace
✅ **Virtual memory** - Address spaces, segments, paging
✅ **Heap management** - malloc/free, memory leaks, patterns

Key Takeaways

Critical Skills

1. **Compile with** `-g -O0` for debugging
2. **Enable core dumps**: `ulimit -c unlimited`
3. **Always check return values** and errno
4. **Pair every malloc() with free()**
5. **Use Valgrind or ASan** to catch memory errors
6. **Profile with perf** before optimizing

Memory bugs are the #1 cause of C crashes

Common Mistakes to Avoid

❌ Ignoring function return values
❌ Not checking malloc() for NULL
❌ Freeing stack variables or string literals
❌ Using uninitialized variables
❌ Buffer overflows (use strncpy, not strcpy)
❌ Forgetting to close file descriptors
❌ Assuming errno is set on success

Test with Valgrind before considering code "done"

Hands-On Exercises

Lab Time: ~4 Hours

Navigate to: `exercises/02-syscalls-memory-debugging.md`

You will:

1. Debug crashes with GDB and core dumps
2. Find memory leaks with Valgrind
3. Trace system calls with strace
4. Write proper error handling code
5. Analyze memory layouts with /proc
6. Profile with perf and address hotspots

Tomorrow: Day 3

File Systems and I/O Operations

- File I/O deep dive (open, read, write, seek)
- File system concepts (inodes, directories)
- Advanced I/O (scatter/gather, async I/O)
- File locking and synchronization
- Working with /proc and /sys

Exercises connect: Use debugging skills while working with file APIs

Questions?

Resources

- GDB Manual: `info gdb`
- Valgrind User Manual: valgrind.org/docs/
- strace man page: `man strace`
- Linux syscall list: `man syscalls`
- errno codes: `man errno`

Office hours: After lab session

Additional Resources

Documentation

- GDB Debugging Tutorial
- Valgrind Quick Start
- AddressSanitizer
- Linux System Call Reference

Books

- "The Linux Programming Interface" by Michael Kerrisk
- "Advanced Programming in the UNIX Environment" by Stevens & Rago

Break!

Next Up: Hands-On Lab

☕ Take 10 minutes, then dive into debugging exercises

Remember: Every expert was once a beginner who didn't give up!